קטע קוד A

```
#include <iostream>
#include <typeinfo>
using namespace std;

template <class T>
class Base
{
   T val;
public:
   Base(const T& t) : val(t)
   {
      cout << "Base: val is " << typeid(val).name() << endl;
   }

   Base(const Base& other) : val(other.val)
   {
      cout << "Base(copy): val is " << typeid(val).name() << endl;
   }

   virtual ~Base()
   {
      cout << "~Base: val is " << typeid(val).name() << endl;
   }

   friend ostream& operator<<(ostream& os, const Base& b)
   {
      os << "I'm {" << typeid(b).name() << "}\n"
         << "val's type = |" << typeid(b.val).name() << "|\n"
         << "val's = [" << b.val << "]\n";

      return os;
   }
};

template<class T>
class Derived : public Base<T>
{
public:
   Derived(const T& t) : Base<T>(t)
   {
      cout << "Derived\n";
   }

   Derived(const Derived& other) : Base<T>(other)
   {
      cout << "Derived(copy)\n";
   }

   ~Derived()
   {
      cout << "~Derived\n";
   }
};
```

להלן מספר main אפשריים:

mainA1

```
void mainA1()
{
   Base<char> b('a');
   Derived<Base<char>> d(b);
   cout << "1-----------------\n";
   cout << d << endl;
   cout << "2-----------------\n";
}
```

mainA2

```
void mainA2()
{
   Base<Base<Base<char>>> b('a');
   cout << "-----------------\n";
}
```

קטע קוד B

```
#include <iostream>
#include <string.h>
using namespace std;

class Sandwich
{
public:
   enum class eBreadType { WHITE, CHIFFON, FULL };
   enum class eStatus { NOT_EATEN, BEING_EATEN, THROWN_AWAY, FINISHED };

   static const char* breadTypes[];
   static const char* statusTypes[];

   Sandwich(const char* spread, eBreadType breadType = eBreadType::FULL);
   Sandwich(const Sandwich& other);
   const Sandwich& operator=(const Sandwich& other);
   const Sandwich& operator=(Sandwich&& other);
   ~Sandwich();

   void setStatus(Sandwich::eStatus newStatus) const { status = newStatus; }

   friend ostream& operator<<(ostream& os, const Sandwich& other);
   operator const char*() const { return spread; }

private:
   char* spread;
   eBreadType breadType;
   mutable eStatus status;
};

const char* Sandwich::breadTypes [] = { "White", "Chiffon", "Full" };
const char* Sandwich::statusTypes[] = { "not eaten", "being eaten", "thrown
away", "finished" };

Sandwich::Sandwich(const char* spread, eBreadType breadType) :
                      breadType(breadType), status(eStatus::NOT_EATEN)
{
   this->spread = strdup(spread);
   cout << "In Sandwich::Sandwich with " << *this << endl;
}

Sandwich::Sandwich(const Sandwich& other) : breadType(other.breadType),
                               status(eStatus::NOT_EATEN)
{
   this->spread = strdup(other.spread);
   cout << "In Sandwich::Sandwich (copy) for " << *this << endl;
}

const Sandwich& Sandwich::operator=(const Sandwich& other)
{
   cout << "In Sandwich::operator= for " << *this << endl;
   if (this != &other)
   {
      delete[] spread;
      spread = strdup(other.spread);
      breadType = other.breadType;
      status = other.status;
   }
   return *this;
}
```

```
const Sandwich& Sandwich::operator=(Sandwich&& other)
{
   cout << "In Sandwich::operator=&& for " << *this << endl;
   if (this != &other)
   {
      delete[] spread;
      spread = other.spread;
      other.spread = nullptr;
      breadType = other.breadType;
      status = other.status;
   }
   return *this;
}

Sandwich::~Sandwich()
{
   cout << "In Sandwich::~Sandwich for " << *this << endl;
   delete[]spread;
}

ostream& operator<<(ostream& os, const Sandwich& other)
{
   if (other.spread)
      os << other.spread;
   else
      os << "NO SPREAD";

   os << " in " << Sandwich::breadTypes[(int)other.breadType]
           << " bread, status: " << Sandwich::statusTypes[(int)other.status];
   return os;
}


class SchoolBag
{
public:
   SchoolBag(const char* name, const Sandwich& sandwich = "cottage");
   SchoolBag(const SchoolBag& other);
   const SchoolBag& operator=(const SchoolBag& other);
   ~SchoolBag();

private:
   char* name;
   Sandwich sandwich;
   // also has some notebooks, but it's not really
   // important as long as there is sandwich in the bag...
};

SchoolBag::SchoolBag(const char* name, const Sandwich& sandwich) :
sandwich(sandwich)
{
   cout << "In SchoolBag::SchoolBag for " << name << "\n";
   this->name = strdup(name);
}

SchoolBag::SchoolBag(const SchoolBag& other) : sandwich(other.sandwich)
{
   cout << "In SchoolBag::SchoolBag (copy) for " << other.name << "\n";
   this->name = strdup(other.name);
}
```

```
const SchoolBag& SchoolBag::operator=(const SchoolBag& other)
{
   cout << "In SchoolBag::operator= for " << name << "\n";
   if (this != &other)
   {
      delete[] name;
      name = strdup(other.name);
      sandwich = other.sandwich;
   }
   return *this;
}

SchoolBag::~SchoolBag()
{
   cout << "In SchoolBag::~SchoolBag for " << name << "\n";
   delete[]name;
}

Sandwich prepareSandwich(const char* spread, Sandwich::eBreadType breadType)
{
   return Sandwich(spread, breadType);
}
```

להלן מספר main אפשריים:

mainB1

```
void mainB1()
{
   Sandwich s1 = "chocolate";
   s1.setStatus(Sandwich::eStatus::BEING_EATEN);

   const Sandwich s2("cheese");
   s2.setStatus(Sandwich::eStatus::BEING_EATEN);

   cout << s1 << endl;
   cout << s2 << endl;
}
```

mainB2

```
void mainB2()
{
   Sandwich s("chocolate");
   SchoolBag sb1("momo", Sandwich("cheese", Sandwich::eBreadType::FULL));
   cout << "1) ----------------------------------\n";
   s = prepareSandwich("hummus", Sandwich::eBreadType::WHITE);
   cout << "2) ----------------------------------\n";
   SchoolBag sb3 = sb1;
   cout << "3) ----------------------------------\n";
}
```

בהצלחה !